1.1 Number Systems

Thursday, 3 July 2025 2:40 pm

3fbbad09-6f8f-4e59-...

1.1 Number Systems

≡ Topic Number	1
Description	This topic covers how different types of data (text, images, sound, etc.) are represented in digital systems using binary code, including number systems, character encoding, and digital media representation.

▼ Index

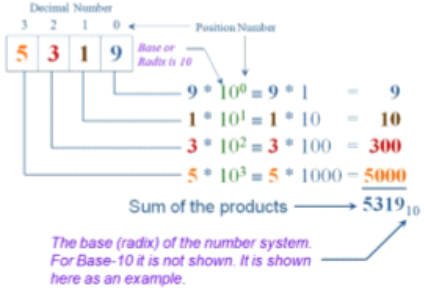
Navigation

▼ Syllabus

Candidates should be able to:	Notes and guidance
1 Understand how and why computers use binary to represent all forms of data	• Any form of data needs to be converted to binary to be processed by a computer • Data is processed using logic gates and stored in registers
2 (a) Understand the denary, binary and hexadecimal number systems	• Denary is a base 10 system • Binary is a base 2 system • Hexadecimal is a base 16 system
(b) Convert between (i) positive denary and positive binary (ii) positive denary and positive hexadecimal (iii) positive hexadecimal and positive binary	• Values used will be integers only • Conversions in both directions, e.g. denary to binary or binary to denary • Maximum binary number length of 16-bit
3 Understand how and why hexadecimal is used as a beneficial method of data representation	• Areas within computer science that hexadecimal is used should be identified • Hexadecimal is easier for humans to understand than binary, as it is a shorter representation of the binary
4 (a) Add two positive 8-bit binary integers (b) Understand the concept of overflow and why it occurs in binary addition	• An overflow error will occur if the value is greater than 255 in an 8-bit register • A computer or a device has a predefined limit that it can represent or store, for example 16-bit • An overflow error occurs when a value outside this limit should be returned
5 Perform a logical binary shift on a positive 8-bit binary integer and understand the effect this has on the positive binary integer	• Perform logical left shifts • Perform logical right shifts • Perform multiple shifts • Bits shifted from the end of the register are lost and zeros are shifted in at the opposite end of the register • The positive binary integer is multiplied or divided according to the shift performed • The most significant bit(s) or least significant bit(s) are lost
6 Use two's complement to represent positive and negative 8-bit binary integers	• Convert a positive binary or denary integer to a two's complement 8-bit integer and vice versa • Convert a negative binary or denary integer to a two's complement 8-bit integer and vice versa

The Denary System

Also known as the **decimal** or Base 10 system. Let's start with deriving some intuition for how our everyday number system actually works



x

But why 10 specifically?

If you look at your hands, you'll understand exactly where the magic number originated from - the 10 fingers on your hands. Since fingers were the most intuitive way of counting, the number system developed around them.

As you can see, every number has a **place value** in the power of ten

The Binary System

The denary system evolved due to our fingers, great - but WHAT IS THIS BINARY SYSTEM AND WHERE DOES IT COME FROM?
Well, as most of you might be aware, computers **do not have fingers.** What they do have are millions of tiny **switches (**resistors**).**

NOT TO BE CONFUSED w/ Resistors

How do registers work?

These registers are electronic devices that detect changes in voltage; a voltage lower than the defined threshold is represented as 0 (In other words, when the switch is off and there is no current flowing through it, the register

How do registers work?

These registers are electronic devices that detect changes in voltage; a voltage lower than the defined threshold is represented as 0 (In other words, when the switch is off and there is no current flowing through it, the register represents a 0). Conversely, a high voltage is represented as 1. **These are the only two states a register can ever be in**

TL/DR: **off = 0 & on = 1**

Since a register can only have **two states: 0 or 1,** it is kind of like it has only two fingers

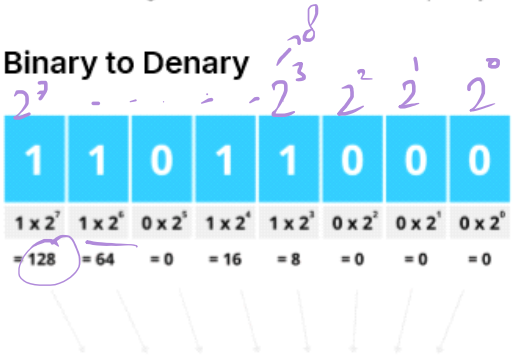
> Resistors are used to store data in computers. Any processing on this data is done using logic gates (see Chapter 10). Therefore, any form of data needs to be converted to binary before it can be processed by a computer

Given how registers are key to a computer's operation, it's only fair that we evolved a number system to facilitate them.

Imagine registers to be an alien being that has only two fingers.

We will once again use the same old concept of **place values.** However, our base this time will be 2

Binary to Denary



128 + 64 + 0 + 16 + 8 + 0 + 0 + 0

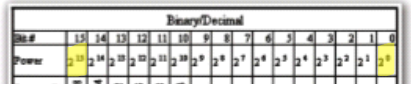
= 216

when left-most bit is on → No will be odd

Binary	Denary
0001	1
0011	3
0111	7
1111	15
1110	14
1100	12
1000	8
0000	0

$2^3\ 2^2\ 2^1\ 2^0$
$8\ 4\ 2\ 1$
$0001 = (1\times1)$
$0011 = (2\times1) + (1\times1)$
$0111 = (4\times1) + (2\times1) + (1\times1)$
$1111 = (8\times1) + (4\times1) + (2\times1) + (1\times1)$
$1110 = 8\times1 + 4\times1 + 2\times1$

Table to Memorize



Your syllabus states that you will be tested on a maximum binary number length of 16-bit.

> Whenever you are counting number of bits.

Binary/Decimal																
Bit #	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Power	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^{9}	2^{8}	2^{7}	2^{6}	2^{5}	2^{4}	2^{3}	2^{2}	2^{1}	2^{0}
Decimal Bit Value	32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1
Max Value	65535

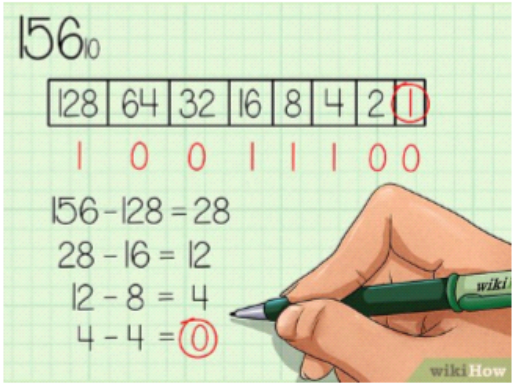
Table 2

binary number length of 16-bit.

> Whenever you are counting number of bits, 0 is included
> As you can see in the attached table, we go up till bit number 15 only. However, since the table
> **starts from bit 0 (as it always should)**, we have a total of 16 bits

The max value you see in the table is the denary value of sixteen 1s (1111111111111111).

Denary to Binary



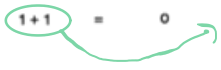
- Turn "on" the bit which has the largest place value less than or equal to your target number.
- Now subtract that bit's value from the target number.
  - The answer to the above subtraction is your new target number.
- Keep repeating the first two steps until you have 0 as your target number

Denary	Binary
2	00000010
5	00000101
9	
13	
56	00111000
255	11111111

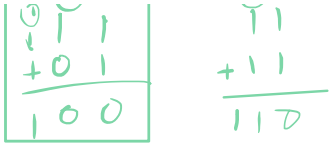
Binary Addition

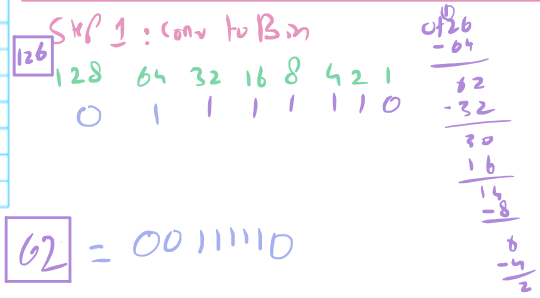
Addition		Result	Carry
0 + 0	=	0	0
0 + 1	=	1	0
1 + 0	=	1	0
1 + 1	=	0	1

00100111 +
01001010
01110001



binary digit	carry	sum
0+0+0	0	0
0+0+1	0	1
0+1+0	0	1
0+1+1	1	0
1+0+0	0	1
1+0+1	1	0
1+1+0	1	0
1+1+1	1	1

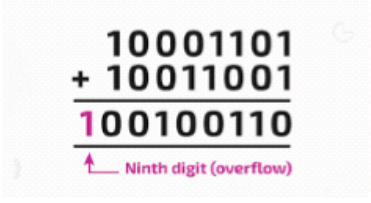




Add in binary 126 + 62

Overflow Error

- Sometimes addition results in more bits that the original numbers
- Overflow indicated that the result of the sum was too large to be stored in the given register size
- This extra bit is known as the overflow bit
- We are forced to ignore the overflow bit, which means that the result of binary addition is now incorrect



> Overflow is an important concept because a computer has a predefined limit on how many bits it can store to represent a number. When a value exceeds this limit, overflow occurs.
> The max value for an 8 bit system is 255

Hexadeximal - 'cause who cares for suchhhh long binaries

With time the aliens evolve. The inconvenience of counting on two fingers resulted them in growing more and more fingers - 16 to be precise.
The same people who had to write a whopping 4 digits to represent 15 (15 in binary is written as 1111) now just need **one character** to represent it.

The hexadecimal number system is very closely related to the binary system. Hexadecimal (sometimes referred to as simply 'hex') is a base 16 system and
therefore needs to use 16 different 'digits' to represent each value.
Because it is a system based on 16 different digits, the numbers 0 to 9 and the
letters A to F are used to represent each hexadecimal (hex)

Binary value	Hexadecimal value	Denary value
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8

Because it is a system based on 16 different digits, the numbers 0 to 9 and the
letters A to F are used to represent each hexadecimal (hex) digit. A in hex = 10
in denary, B = 11, C = 12, D = 13, E = 14 and F = 15.

0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	A	10
1011	B	11
1100	C	12
1101	D	13
1110	E	14
1111	F	15

> Since 16 is 2^4, one hexadecimal number is equivalent to 4 binary numbers (you can convince yourself by seeing that the biggest hex number, 15, is 4 bit).

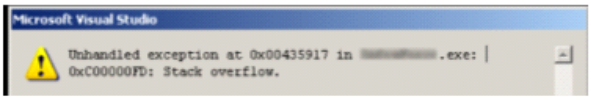
> Hexadecimal is easier for humans to understand than binary, as it is a shorter representation of the binary

Whilst computer scientists can work with binary, they find hexadecimal to be more convenient to use. This is because one hex digit represents four binary digits. A complex
binary number, such as 1101001010101111 can be written in hex as D2AF. The hex number is far easier for humans to remember, write, copy and work with.

Uses of Hexadecimal

Error Codes

Error codes are often shown as hexadecimal values. These numbers refer to the memory location of the error and are usually automatically generated by the computer.



Media Access Control (MAC) addresses

MAC address refers to a number which uniquely identifies a device on a network. The MAC address refers to the network interface card (NIC) which is part of the device. The MAC address is rarely changed so that a particular device can always be identified no matter where it is. A MAC address is usually made up of 48 bits which are shown as 6 groups of two hexadecimal digits

Internet Protocol (IP) addresses

Each device connected to a network is given an address known as the IP address. An IPv6 address is a 128-bit number broken down into 16-bit chunks, represented by a hexadecimal number.

a8fb:7a88:fff0:0fff:3d21:2085:66fb:f0fa

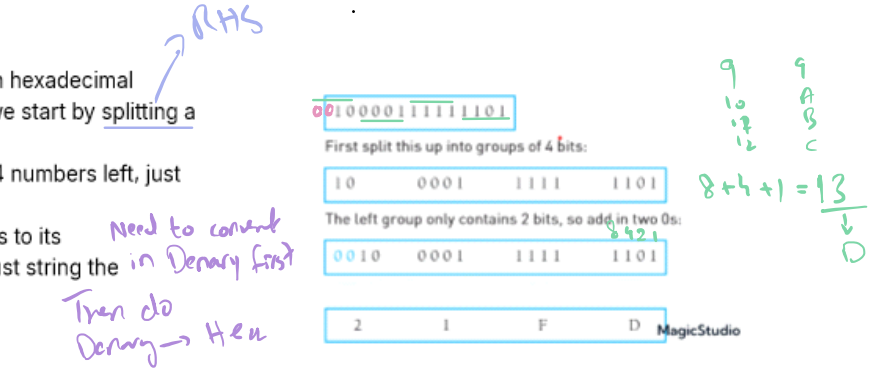
HyperText Mark-up Language (HTML) colour codes

HTML isn't a programming language but is simply a mark-up language. A mark-up language is used in the processing, definition and presentation of text and is used in webpage development.
HTML is often used to represent colours of text on the computer screen. All colours can be made up of different combinations of the three primary colours (red, green and blue). The different intensity of each colour (red, green and blue) is determined by its hexadecimal value. This means different hexadecimal values represent different colours.

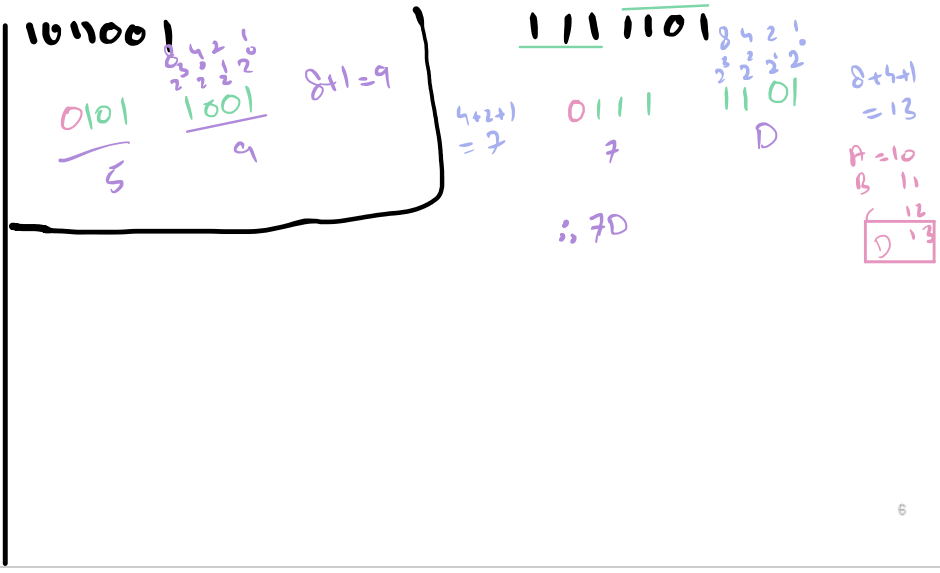
Binary to Hexadecimal

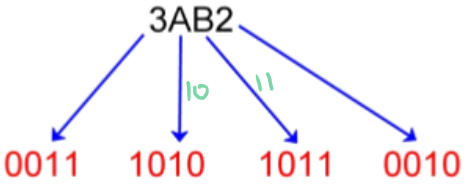
Binary to Hexadecimal

The process is fairly simple. Since each hexadecimal number is made up of 4 binary digits, we start by splitting a binary number into groups of 4.
If in the last group, there are less than 4 numbers left, just append zeros to the left side
Finally, just convert each group of 4 bits to its corresponding hexadecimal and then just string the numbers together



Hexadecimal to Binary





$3AB2_{16} = 11101010110010_2$

Way too simple

Left Shift by 2

0x11 00

Right Shift by 1

0101 0

Logical Binary Shift

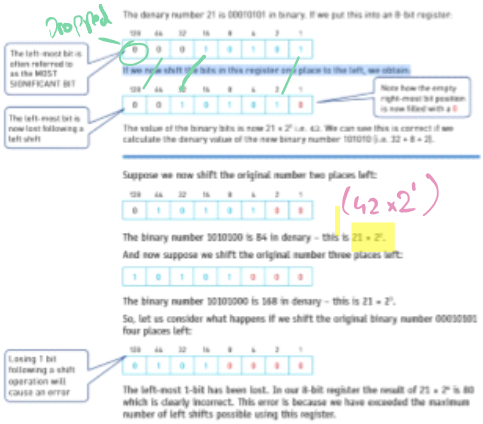
Logical shifts move binary numbers left or right, where:

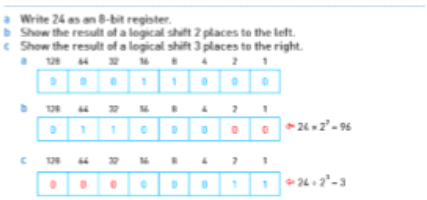
- **Each left shift** = multiply by 2
- **Each right shift** = divide by 2

Bits shifted from the end of the register are lost, and zeros are shifted in at the opposite end of the register.
In the case of a left shift, we essentially remove the left-most bits (also called the **most significant bit**) and add a zero to the right of our number, making sure the number of bits remains the same.
Conversely, the right-most (**least significant bits**) are removed when right shifting and zeros are added on the left side of the number.

Eventually, after a number of shifts, the register would only contain zeros. For example, if we shift 01110000 (denary value 112) five places left (the equivalent to multiplying by 25 , i.e. 32), in an 8-bit register we would end up with 00000000. This makes it seem as though 112 × 32 = 0! This would result in the generation of an error message.

Eventually, after a number of shifts, the register would only contain zeros. For example, if we shift 01110000 (denary value 112) five places left (the equivalent to multiplying by 25 , i.e. 32), in an 8-bit register we would end up with 00000000. This makes it seem as though 112 × 32 = 0! This would result in the generation of an error message.



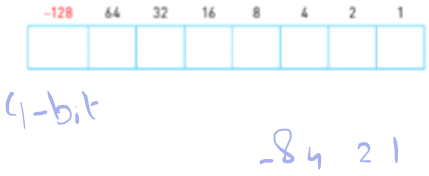


Images taken from O Level Computer Science Second Edition - Hodder

Two's Complement

Similar to our emotions, not all numbers are positive. To allow the possibility of representing negative integers, we make use of two's complement.

Only one small change is made for two's complement representation of a number: the **most significant bit** is changed to a negative value. In an 8-bit number, it looks something like this:



> Two's complement can be used to represent **both** positive and negative numbers - not just negative
> When representing a positive number, we make sure to have a 0 in the most significant bit, while we put a 1 (or
> *turn the bit on*) there when representing a negative number.

Positive Denary to Two's Complement

38

-128	64	32	16	8	4	2	1
0	0	1	0	0	1	1	0

125

-128	64	32	16	8	4	2	1
0	1	1	1	1	1	0	1

Positive Binary to Two's Complement

01101110

Positive Binary to Two's Complement

01101110

-128	64	32	16	8	4	2	1
0	1	1	0	1	1	1	0

Converting positive numbers in two's complement representation is similar to normal binary representation. You just need to make sure that the **most significant bit remains zero.**

Does this mean that you can no longer represent numbers like 255 in binary?
Well, Yes. A trade-off of using two's complement representation is that when representing positive numbers, you effectively have only 7 bits when using an 8 bit register (as the most significant bit is now the sign bit and remains off)

Negative Denary to Two's Complement

-6

-128	64	32	16	8	4	2	1
1	1	1	1	1	0	1	0

-128 + 64 + 32 + 16 + 8 + 2 = -6

Denary	TC
-97	10011111
-42	11010110
-32	
-128	10000000

STEP 1: Write down the place values with Most Significant bit as negative
STEP 2: For Neg Nums, ALWAYS turn on the MSB
STEP3: Moving left to right, check for each bit if you need to turn that bit on or not in order to get the target number

37 ⟶ -37

Positive Binary to Negative Binary (Two's Complement)

To represent -34 in 2's complement form

+ 34 = 0 0 1 0 0 0 1 0

1 0 — Copy all bits till first '1' is encountered in the number

1 1 0 1 1 1 1 0 — Invert all 1s with 0s and 0s with 1s then after

- 34 = 1 1 0 1 1 1 1 0 — 2's Complement of +34

- 34 = 1 1 0 1 1 1 1 0 2's Complement of +34